

gemstone-rs Examples Guide

A tour of Rust examples and tooling workflows



Generated from the Markdown sources in `gemstone-rs/docs`.

Contents

Examples Guide

Common Setup

Example Map

Suggested Learning Order

Expected Output

CLI Equivalents

Codegen Workflow

VS Code

Later Examples

Examples Guide

The `gemstone-rs` examples are split between runnable Cargo examples and the checked-in codegen sample under the repository-level `examples/` directory.

Common Setup

```
gemstone-rs env sample
gemstone-rs env write
gemstone-rs doctor --env-file .env.gemstone-rs
gemstone-rs --env-file .env.gemstone-rs browse dictionaries
gemstone-rs --env-file .env.gemstone-rs codegen check examples/codegen/gemstone-rs.codegen
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

Run Cargo examples from the repository root:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example quickstart
```

Example Map

Feature	Command or path	What it demonstrates
First login	<code>cargo run -p gemstone-rs --example hello_gemstone</code>	Reads env config, logs in, prints a session id, and evaluates <code>3 + 4</code> .
Quickstart	<code>cargo run -p gemstone-rs --example quickstart</code>	Eval, <code>global_put</code> , <code>global_get</code> , string fetch, cleanup.
Eval only	<code>cargo run -p gemstone-rs --example eval</code>	Minimal <code>Session::eval</code> shape.
Browser API	<code>cargo run -p gemstone-rs --example browser</code>	Dictionaries, protocols, methods, and source.

Feature	Command or path	What it demonstrates
Live smoke cookbook	<code>cargo run -p gemstone-rs -- example live_smoke_cookbook</code>	Login, eval, global round-trip, perform, and transaction checks in one run.
Transactions	<code>cargo run -p gemstone-rs -- example transactions</code>	Commit-on-success and abort-on-error transaction wrapper.
OOP/value conversion	<code>cargo run -p gemstone-rs -- example oop_values</code>	<code>Value</code> , <code>Oop</code> , strings, symbols, and export-set retention.
BridgeRoot mapping	<code>cargo run -p gemstone-rs -- example bridge_root_mapping</code>	MagLev-style bridge-root storage with explicit <code>BridgeValue</code> mapping.
Derive mapping	<code>cargo run -p gemstone-rs -- example derive_mapping</code>	<code>#[derive(BridgeMapped)]</code> , symbol keys, nested structs, vectors, maps, and BridgeRoot transactions.
Offline codegen	<code>cargo run -p gemstone-rs -- example codegen_preview</code>	Generates wrappers from the sample config without a live stone.
Codegen workflow	<code>cargo run -p gemstone-rs -- example codegen_workflow</code>	Writes config, previews, diffs, checks, generates, and verifies a clean diff.
Codegen discovery	<code>cargo run -p gemstone-rs -- example codegen_discover</code>	Connects to a live stone and discovers a starter config for <code>Object</code> .
Mapping discovery	<code>cargo run -p gemstone-rs -- example codegen_discover_mapping</code>	Connects to a live stone and proposes a <code>BridgeMapped</code> config.
Generated wrapper app	<code>cargo run -p gemstone-rs -- example generated_wrapper_app</code>	Uses checked-in generated wrappers to call <code>Object>>printString</code> .
Generated mapping app	<code>cargo run -p gemstone-rs -- example generated_mapping_app</code>	Uses codegen-created <code>BridgeMapped</code> structs with <code>BridgeRoot</code> .
Generated wrapper compile check	<code>cargo test --manifest-path examples/codegen-wrapper-check/ Cargo.toml</code>	Imports the checked-in generated wrappers as a separate crate.
Codegen files	<code>examples/codegen/</code>	Config, generated wrappers, check/diff/generate workflow.
Explorer tooling	<code>examples/tooling/explorer.md</code>	Local explorer startup and endpoint checks.
VS Code tooling		

Feature	Command or path	What it demonstrates
	<code>examples/tooling/vscode-workbench.md</code>	Sidebar browsing, codegen actions, and explorer launch.
CLI browser walkthrough	<code>examples/tooling/cli-browser-walkthrough.md</code>	Terminal-only browse workflow.
Axum service sketch	<code>examples/axum-service/README.md</code>	Recommended web-service shape without adding workspace dependencies.

Suggested Learning Order

1. `hello_gemstone`
2. `quickstart`
3. `browser`
4. `live_smoke_cookbook`
5. `oop_values`
6. `transactions`
7. `bridge_root_mapping`
8. `derive_mapping`
9. `codegen_preview`
10. `codegen_workflow`
11. `generated_wrapper_app`
12. `generated_mapping_app`
13. `codegen_discover`
14. `codegen_discover_mapping`
15. `examples/codegen/`
16. `examples/tooling/cli-browser-walkthrough.md`
17. `examples/tooling/explorer.md`
18. `examples/tooling/vscode-workbench.md`
19. `examples/axum-service/README.md`

Expected Output

Live examples need GemStone credentials and a reachable stone. If the environment is configured correctly, these are the important lines to look for:

```
$ cargo run -p gemstone-rs --example hello_gemstone
session id: <number>
3 + 4 => SmallInt(7)

$ cargo run -p gemstone-rs --example quickstart
GemStone eval ok: SmallInt(7)
```

```
GemStoneRsQuickstart: hello from gemstone-rs quickstart

$ cargo run -p gemstone-rs --example generated_wrapper_app
generated wrapper printString: 7

$ cargo run -p gemstone-rs --example live_smoke_cookbook
login ok: session <number>
eval ok: SmallInt(7)
global round-trip ok
perform ok: 7
transaction commit/abort ok

$ cargo run -p gemstone-rs --example bridge_root_mapping
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP", labels:
{"source": "manual"} }
loaded status: ready
loaded amount: 100
loaded approved: true
loaded tags: ["priority", "demo"]
loaded note: Some("front desk")
loaded labels: {"source": "manual"}
loaded symbol labels: {"source": "manual"}

$ cargo run -p gemstone-rs --example derive_mapping
derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"], labels: {"source": "derive"}, note: None }

$ cargo run -p gemstone-rs --example generated_mapping_app
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"], labels: {"source": "generated"}, note: Some("window
seat") }
```

Offline examples should run without GemStone:

```
$ cargo run -p gemstone-rs --example codegen_workflow
before generate: exists=false up_to_date=false diff_bytes=<number>
after generate: exists=true up_to_date=true
diff after generate: clean
```

CLI Equivalents

The CLI gives you the same live checks without compiling examples:

```
cargo install gemstone-rs-cli
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --strict
gemstone-rs doctor --json
```

```

gemstone-rs eval --env-file .env.gemstone-rs "3 + 4"
gemstone-rs browse dictionaries
gemstone-rs browse classes UserGlobals
gemstone-rs browse protocols Object
gemstone-rs browse methods Object "-- all --"
gemstone-rs browse source Object printString
gemstone-rs inspect oop 20
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge sample-config BookingDraft
gemstone-rs codegen explain examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain --json examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain-profile --json default examples/codegen/gemstone-
rs.codegen-profiles.json
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml

```

Codegen Workflow

```

cargo run -p gemstone-rs-cli -- codegen init examples/codegen/demo.codegen
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen diff examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen check examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen discover-mapping examples/codegen/
mapping.codegen BookingDraft Object

```

Use the checked-in explorer profile sample when you want a repeatable browser workflow for codegen:

```

cat examples/codegen/gemstone-rs.codegen-profiles.json
gemstone-rs-explorer --port 8787 --codegen-root .

```

In the explorer, click **Load Project Profiles** with **examples/codegen/gemstone-rs.codegen-profiles.json** as the project profile file, then select **default**, **object-wrapper**, or **bridge-mapping**.

Validate profile files before committing them:

```

cargo run -p gemstone-rs-cli -- profile validate examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile check examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json

```

Generate a starter config from a live stone:

```
cargo run -p gemstone-rs-cli -- codegen discover examples/codegen/discovered.codegen
Object
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated wrapper example after checking the generated file into the repository:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

VS Code

Install the workbench from:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Use the GemStone RS sidebar to browse dictionaries, classes, protocols, and methods. The same sidebar exposes Codegen Discover, Preview, Diff, Check, Generate, profile-driven codegen actions, Generate Mapping Config, Preview BridgeRoot, List BridgeRoot Keys, Put BridgeRoot String, Remove BridgeRoot Key, Run Generated Mapping Example, Load Project Profiles, Save Project Profiles, Export Codegen Profile, Show Sample Project Profiles, Create Project Profiles, Validate Project Profiles, List Project Profiles, Show Project Profile, and Resolve Project Profile, Check Project Profiles, and Open Docs actions.

Later Examples

These are useful, but should wait until the corresponding APIs are stable:

- a local explorer workflow with screenshots
- a full Axum or Actix workspace member wired into CI
- a richer class browser walkthrough with captured output

This PDF was generated locally from docs/. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.